

**BRACT's**

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI      Semester: 5      Academic Year: 2026-27

Division: E      Batch: 3      Practical No: 2

Roll no: 62      PRN No: 12413457      Name of Student: Swaraj Shedge

**Subject Name : Deep Learning**

## **Problem Statement :**

Implement a Single-Layer Perceptron to perform binary classification using Python. Train the perceptron on a simple dataset and analyze how changing the weights, bias, and activation function affects the classification results.

## **Algorithm :**

1. Import the required Python libraries.
2. Create a binary classification dataset with two input features.
3. Define the step activation function.
4. Initialize the perceptron with random weights and bias.
5. For each training epoch:
  - Calculate the weighted sum.
  - Apply the activation function.
  - Compute the prediction error.
  - Update the weights using the perceptron learning rule.
  - Update the bias.

6. Repeat until all epochs are completed.
7. Display the final weights and bias.
8. Test the perceptron on the training data.
9. Change the weights and observe the output.
10. Change the bias and observe the output.
11. Replace the activation function (e.g., Sigmoid instead of Step) and compare the outputs.
12. Analyze how weights, bias, and activation functions influence the classification.

## Code / Output:

```
import numpy as np

[1] ✓ 5.6s

x = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 0, 0, 1])

print("Input Data:")
print(x)

print("\nTarget Labels:")
print(y)

[2] ✓ 0.0s

... Input Data:
[[0 0]
 [0 1]
 [1 0]
 [1 1]]

Target Labels:
[0 0 0 1]
```

```
def step_function(x):
    if x >= 0:
        return 1
    return 0

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

[3] ✓ 0.0s

```
weights = np.random.rand(2)
bias = np.random.rand()

learning_rate = 0.1
epochs = 10

print("Initial Weights:", weights)
print("Initial Bias:", bias)
```

[4] ✓ 1.3s

... Initial Weights: [0.14269218 0.23385853]  
Initial Bias: 0.3504235098824049

```
for epoch in range(epochs):
    for i in range(len(X)):
        weighted_sum = np.dot(X[i], weights) + bias
        prediction = step_function(weighted_sum)
        error = y[i] - prediction
        weights = weights + learning_rate * error * X[i]
        bias = bias + learning_rate * error

    print("Training Completed")
    print("Final Weights:", weights)
    print("Final Bias:", bias)
```

[5] ✓ 0.0s

... Training Completed  
Final Weights: [0.14269218 0.03385853]  
Final Bias: -0.1495764901175951

```
print("Predictions using Step Activation Function")

for i in range(len(X)):
    weighted_sum = np.dot(X[i], weights) + bias
    prediction = step_function(weighted_sum)
    print(f"Input: {X[i]} Prediction: {prediction}")
```

[6] ✓ 0.0s

```
... Predictions using Step Activation Function
Input: [0 0] Prediction: 0
Input: [0 1] Prediction: 0
Input: [1 0] Prediction: 0
Input: [1 1] Prediction: 1
```

```
print("Effect of Changing Weights")

new_weights = np.array([2.0, 2.0])

for i in range(len(X)):
    weighted_sum = np.dot(X[i], new_weights) + bias
    prediction = step_function(weighted_sum)
    print(f"Input: {X[i]} Prediction: {prediction}")
```

[7] ✓ 0.0s

```
... Effect of Changing Weights
Input: [0 0] Prediction: 0
Input: [0 1] Prediction: 1
Input: [1 0] Prediction: 1
Input: [1 1] Prediction: 1
```

```

print("Effect of Changing Bias")

new_bias = -2

for i in range(len(X)):
    weighted_sum = np.dot(X[i], weights) + new_bias
    prediction = step_function(weighted_sum)
    print(f"Input: {X[i]} Prediction: {prediction}")

```

[8] ✓ 0.0s

```

... Effect of Changing Bias
Input: [0 0] Prediction: 0
Input: [0 1] Prediction: 0
Input: [1 0] Prediction: 0
Input: [1 1] Prediction: 0

```

```

print("Predictions using Sigmoid Activation Function")

for i in range(len(X)):
    weighted_sum = np.dot(X[i], weights) + bias
    prediction = sigmoid(weighted_sum)
    print(f"Input: {X[i]} Output: {prediction:.4f}")

```

[9] ✓ 0.0s

```

... Predictions using Sigmoid Activation Function
Input: [0 0] Output: 0.4627
Input: [0 1] Output: 0.4711
Input: [1 0] Output: 0.4983
Input: [1 1] Output: 0.5067

```